



UAI301 Object Oriented Programming

Lab Assignment 3

Name: Amritanshu Mishra

Roll No.: 1025250008

Sub-Group: 2X11

Subject Code: UAI301

Subject Name: Object Oriented Programming

Question 1 Create a class named Student. Create an object of the class and display a message: "Welcome to Object-Oriented Programming using Python".

```
In [1]: class Student1:
        def display(self):
            print("Welcome to Object-Oriented Programming using Python")

s1 = Student1()
s1.display()
```

Welcome to Object-Oriented Programming using Python

Question 2 Create a class Student having the following attributes: Name, Branch, and Semester. Initialise the attributes using a constructor and display the details using a member function.

```
In [2]: class Student2:
        def __init__(self, name, branch, semester):
            self.name = name
            self.branch = branch
            self.semester = semester

        def display(self):
            print(f"Name: {self.name}")
            print(f"Branch: {self.branch}")
            print(f"Semester: {self.semester}")

s2 = Student2("Amanpreet Kaur", "CSE", 5)
s2.display()
```

Name: Amanpreet Kaur
Branch: CSE
Semester: 5

Question 3 Create a class Employee with attributes: Employee ID, Employee Name, and Salary. Display the details of two employees.

```
In [3]: class Employee:
        def __init__(self, emp_id, name, salary):
            self.emp_id = emp_id
            self.name = name
            self.salary = salary

        def display(self):
            print(f"ID: {self.emp_id}, Name: {self.name}, Salary: {self.salary}")

e1 = Employee(101, "Rohan Sharma", 45000)
e2 = Employee(102, "Priya Singh", 52000)
e1.display()
e2.display()
```

```
ID: 101, Name: Rohan Sharma, Salary: 45000
ID: 102, Name: Priya Singh, Salary: 52000
```

Question 4 Create a class Rectangle. The constructor should accept: Length and Breadth. Create a function to calculate the area.

```
In [4]: class Rectangle:
        def __init__(self, length, breadth):
            self.length = length
            self.breadth = breadth

        def area(self):
            return self.length * self.breadth

r = Rectangle(10, 5)
print(f"Area of Rectangle: {r.area()}")
```

```
Area of Rectangle: 50
```

Question 5 Create a class Circle. Input the radius through the constructor and calculate the area of the circle. (Use $\pi = 3.14$)

```
In [5]: class Circle:
        PI = 3.14

        def __init__(self, radius):
            self.radius = radius

        def area(self):
            return Circle.PI * self.radius ** 2

c = Circle(7)
```

```
print(f"Area of Circle: {c.area()}")
```

Area of Circle: 153.86

Question 6 Create a class BankAccount. The class should have: Account Number, Account Holder Name, and Balance. Create methods: Deposit, Withdraw, and Display Balance.

```
In [6]: class BankAccount:
        def __init__(self, acc_number, holder_name, balance=0):
            self.acc_number = acc_number
            self.holder_name = holder_name
            self.balance = balance

        def deposit(self, amount):
            self.balance += amount
            print(f"Deposited {amount}. New Balance: {self.balance}")

        def withdraw(self, amount):
            if amount > self.balance:
                print("Insufficient balance!")
            else:
                self.balance -= amount
                print(f"Withdrew {amount}. New Balance: {self.balance}")

        def display_balance(self):
            print(f"Account No: {self.acc_number}, Holder: {self.holder_name}")

acc = BankAccount(12345, "Karan Mehta", 1000)
acc.display_balance()
acc.deposit(500)
acc.withdraw(300)
acc.withdraw(5000)
acc.display_balance()
```

Account No: 12345, Holder: Karan Mehta, Balance: 1000

Deposited 500. New Balance: 1500

Withdrew 300. New Balance: 1200

Insufficient balance!

Account No: 12345, Holder: Karan Mehta, Balance: 1200

Question 7 Create a class Book. Store: Book ID, Title, and Author. Create three objects and display their details.

```
In [7]: class Book:
        def __init__(self, book_id, title, author):
            self.book_id = book_id
            self.title = title
            self.author = author

        def display(self):
            print(f"ID: {self.book_id}, Title: {self.title}, Author: {self.au
```

```

b1 = Book(1, "Python Basics", "John Smith")
b2 = Book(2, "Data Structures", "Jane Doe")
b3 = Book(3, "Operating Systems", "R. K. Sharma")
for b in (b1, b2, b3):
    b.display()

```

ID: 1, Title: Python Basics, Author: John Smith
ID: 2, Title: Data Structures, Author: Jane Doe
ID: 3, Title: Operating Systems, Author: R. K. Sharma

Question 8 Create a class Student. Input: Name, and Marks in three subjects. Create a method to calculate: Total Marks, Percentage.

```

In [8]: class Student8:
        def __init__(self, name, marks1, marks2, marks3):
            self.name = name
            self.marks1 = marks1
            self.marks2 = marks2
            self.marks3 = marks3

        def total_marks(self):
            return self.marks1 + self.marks2 + self.marks3

        def percentage(self):
            return (self.total_marks() / 300) * 100

        def display(self):
            print(f"Name: {self.name}")
            print(f"Total Marks: {self.total_marks()}")
            print(f"Percentage: {self.percentage():.2f}%")

s8 = Student8("Neha Verma", 85, 90, 78)
s8.display()

```

Name: Neha Verma
Total Marks: 253
Percentage: 84.33%

Question 9 Create a class Laptop with attributes: Brand, Processor, RAM and Price. Create a method to display all specifications.

```

In [9]: class Laptop:
        def __init__(self, brand, processor, ram, price):
            self.brand = brand
            self.processor = processor
            self.ram = ram
            self.price = price

        def display_specs(self):
            print(f"Brand: {self.brand}")

```

```
print(f"Processor: {self.processor}")
print(f"RAM: {self.ram}")
print(f"Price: {self.price}")
```

```
laptop = Laptop("Dell", "Intel i7", "16GB", 75000)
laptop.display_specs()
```

Brand: Dell
Processor: Intel i7
RAM: 16GB
Price: 75000

Question 10 Create a class LibraryBook. Store: Book Name, Author, and Number of Copies. Create methods: Issue Book (decrease copies by 1 if available), Return Book (increase copies by 1), and Display Details.

```
In [10... class LibraryBook:
    def __init__(self, book_name, author, copies):
        self.book_name = book_name
        self.author = author
        self.copies = copies

    def issue_book(self):
        if self.copies > 0:
            self.copies -= 1
            print(f"'{self.book_name}' issued. Copies left: {self.copies}")
        else:
            print(f"'{self.book_name}' is not available.")

    def return_book(self):
        self.copies += 1
        print(f"'{self.book_name}' returned. Copies now: {self.copies}")

    def display_details(self):
        print(f"Book: {self.book_name}, Author: {self.author}, Copies: {s

lb = LibraryBook("Wings of Fire", "A. P. J. Abdul Kalam", 3)
lb.display_details()
lb.issue_book()
lb.return_book()
```

Book: Wings of Fire, Author: A. P. J. Abdul Kalam, Copies: 3
'Wings of Fire' issued. Copies left: 2
'Wings of Fire' returned. Copies now: 3

Question 11 Create a class Calculator with methods: add(), subtract(), multiply(), and divide(). The numbers should be provided through the constructor.

```
In [11... class Calculator:
    def __init__(self, num1, num2):
```

```

        self.num1 = num1
        self.num2 = num2

    def add(self):
        return self.num1 + self.num2

    def subtract(self):
        return self.num1 - self.num2

    def multiply(self):
        return self.num1 * self.num2

    def divide(self):
        if self.num2 == 0:
            return "Cannot divide by zero"
        return self.num1 / self.num2

calc = Calculator(20, 4)
print(f"Add: {calc.add()}")
print(f"Subtract: {calc.subtract()}")
print(f"Multiply: {calc.multiply()}")
print(f"Divide: {calc.divide()}")

```

Add: 24
Subtract: 16
Multiply: 80
Divide: 5.0

Question 12 Create a Car class with attributes brand, model, year, and price. Use a constructor to initialize the attributes. Create methods to display the car details and update the price.

In [12...

```

class Car:
    def __init__(self, brand, model, year, price):
        self.brand = brand
        self.model = model
        self.year = year
        self.price = price

    def display_details(self):
        print(f"Brand: {self.brand}, Model: {self.model}, Year: {self.yea

    def update_price(self, new_price):
        self.price = new_price
        print(f"Price updated to: {self.price}")

car = Car("Tata", "Nexon", 2023, 1400000)
car.display_details()
car.update_price(1350000)

```

Brand: Tata, Model: Nexon, Year: 2023, Price: 1400000
Price updated to: 1350000

Question 13 Create a MobilePhone class with attributes brand, model, storage, and battery_percentage. Use a constructor to initialize the details. Implement methods to display phone information, charge the battery, and check the battery status.

```
In [13... class MobilePhone:
    def __init__(self, brand, model, storage, battery_percentage):
        self.brand = brand
        self.model = model
        self.storage = storage
        self.battery_percentage = battery_percentage

    def display_info(self):
        print(f"Brand: {self.brand}, Model: {self.model}, Storage: {self.
              f"Battery: {self.battery_percentage}%"")

    def charge_battery(self, amount):
        self.battery_percentage = min(100, self.battery_percentage + amou
        print(f"Battery charged. Current level: {self.battery_percentage}")

    def check_battery_status(self):
        if self.battery_percentage <= 20:
            print("Battery status: Low, please charge soon.")
        elif self.battery_percentage <= 80:
            print("Battery status: Moderate.")
        else:
            print("Battery status: Good.")

phone = MobilePhone("Samsung", "Galaxy S23", "128GB", 15)
phone.display_info()
phone.check_battery_status()
phone.charge_battery(50)
phone.check_battery_status()
```

```
Brand: Samsung, Model: Galaxy S23, Storage: 128GB, Battery: 15%
Battery status: Low, please charge soon.
Battery charged. Current level: 65%
Battery status: Moderate.
```

Question 14 Create a Product class with attributes product_name, product_id, price, and quantity. Implement methods to calculate the total bill and display product details.

```
In [14... class Product:
    def __init__(self, product_name, product_id, price, quantity):
        self.product_name = product_name
        self.product_id = product_id
        self.price = price
        self.quantity = quantity
```

```

def total_bill(self):
    return self.price * self.quantity

def display_details(self):
    print(f"Product: {self.product_name}, ID: {self.product_id}, "
          f"Price: {self.price}, Quantity: {self.quantity}")
    print(f"Total Bill: {self.total_bill()}")

product = Product("Notebook", "P001", 50, 4)
product.display_details()

```

Product: Notebook, ID: P001, Price: 50, Quantity: 4
Total Bill: 200

Question 15 Create a BookStore class with attributes book_name, author, price, and number_of_copies. Implement methods to display book details, sell a book, and add new copies.

In [15...

```

class BookStore:
    def __init__(self, book_name, author, price, number_of_copies):
        self.book_name = book_name
        self.author = author
        self.price = price
        self.number_of_copies = number_of_copies

    def display_details(self):
        print(f"Book: {self.book_name}, Author: {self.author}, "
              f"Price: {self.price}, Copies: {self.number_of_copies}")

    def sell_book(self, qty=1):
        if self.number_of_copies >= qty:
            self.number_of_copies -= qty
            print(f"Sold {qty} copy/copies. Remaining: {self.number_of_co
        else:
            print("Not enough copies to sell.")

    def add_copies(self, qty):
        self.number_of_copies += qty
        print(f"Added {qty} copies. Total now: {self.number_of_copies}")

store = BookStore("The Alchemist", "Paulo Coelho", 250, 10)
store.display_details()
store.sell_book(3)
store.add_copies(5)

```

Book: The Alchemist, Author: Paulo Coelho, Price: 250, Copies: 10
Sold 3 copy/copies. Remaining: 7
Added 5 copies. Total now: 12

Question 16 Create an Attendance class with attributes student_name, total_classes, and attended_classes. Implement methods to calculate the

attendance percentage and display whether the student is eligible to appear in the examination.

```
In [16... class Attendance:
    def __init__(self, student_name, total_classes, attended_classes):
        self.student_name = student_name
        self.total_classes = total_classes
        self.attended_classes = attended_classes

    def attendance_percentage(self):
        return (self.attended_classes / self.total_classes) * 100

    def display_eligibility(self):
        percentage = self.attendance_percentage()
        print(f"Student: {self.student_name}, Attendance: {percentage:.2f}%")
        if percentage >= 75:
            print("Eligible to appear in the examination.")
        else:
            print("Not eligible to appear in the examination.")

att = Attendance("Simran Kaur", 100, 80)
att.display_eligibility()
```

```
Student: Simran Kaur, Attendance: 80.00%
Eligible to appear in the examination.
```